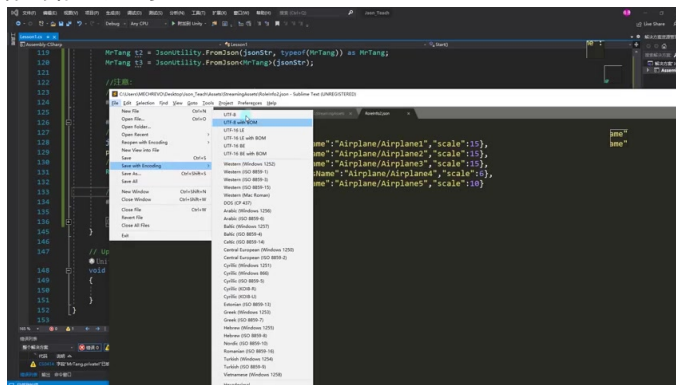


- 容错机制：
 - JSON中缺少的字段不会报错
 - 对应类成员会保持默认值
 - 如Dictionary字段会变为null

2) 数据集处理 06:09

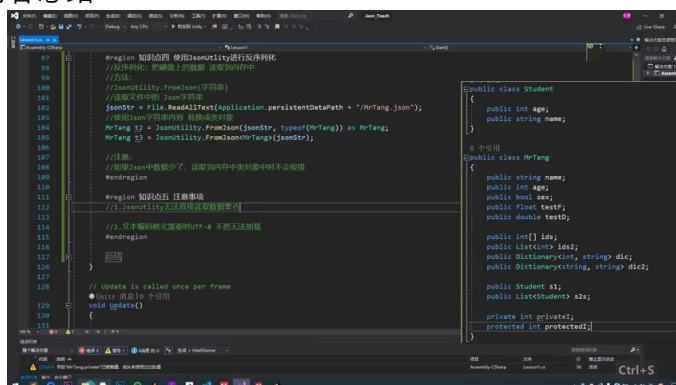
- 限制：无法直接解析纯数组结构的JSON
- 解决方案：
 - 将数组包裹在对象中
 - 创建对应的包装类

3) 编码格式要求



- 必须使用UTF-8编码：
 - 其他编码格式会导致解析失败
 - 常见编辑器可修改文件编码
- 验证方法：
 - 检查文本编辑器编码设置
 - 确保保存为"UTF-8 without BOM"格式

3. 内容总结 14:04



- 核心功能：
 - 轻量级的JSON序列化/反序列化
 - Unity内置无需额外插件
- 适用场景：
 - 简单数据结构存储
 - 需要快速实现的序列化需求
- 局限性：
 - 不支持复杂数据结构
 - 功能相比第三方库较基础
- 最佳实践：
 - 优先使用泛型方法
 - 注意类定义规范
 - 确保文件编码正确

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
序列化与反序列化	序列化将内存对象转为硬盘数据 (JSON字符串), 反序列化将JSON字符串还原为内存对象	JsonUtility.ToJson (序列化) 与 JsonUtility.FromJson (反序列化) 的用法差异	★★
JSON文件读写	使用File.ReadAllText读取文件内容, File.WriteAllText写入JSON字符串	文件路径需正确, 编码格式必须为 UTF-8	★★
自定义类序列化	需添加 [System.Serializable] 特性, 私有/保护变量需加 [SerializeField]	嵌套类需逐层标记序列化特性	★★★
字典与数据集合限制	不支持字典序列化, 数据集合 (如List) 需包裹在父类中反序列化	直接反序列化数组会报错, 需改为 { "list": [...] } 结构	★★★ ★
浮点数精度问题	序列化浮点数可能存在误差 (如1.4存为1.39999), 但反序列化后恢复原值	实际测试中未影响结果, 但需注意精度敏感场景	★★
编码格式要求	JSON文件必须为 UTF-8 编码, 其他格式 (如UTF-16) 会导致解析失败	报错时优先检查编码格式	★
数据缺失处理	JSON中缺少字段时, 反序列化后对象对应字段为默认值 (如字典为null)	不会报错 , 但需手动处理缺失逻辑	★★